

Relatório de Entrega: CI/CD, Testes Unitários e Automação

Disciplina de Integração e Entrega Contínua (DevOps)

Aluno: João Meneses (meneeses)	Projeto: telegram-bot-ci (Bot Node.js)
Repositório: https://github.com/meneeses/ci-cd-bot-project	Status: 13 Testes Unitários Aprovados • CI & Alertas OK

1. Atividade da Semana 6 - Arquivo de Workflow de Alertas

ARQUIVO: .GITHUB/WORKFLOWS/DISCORD-ALERT.YML

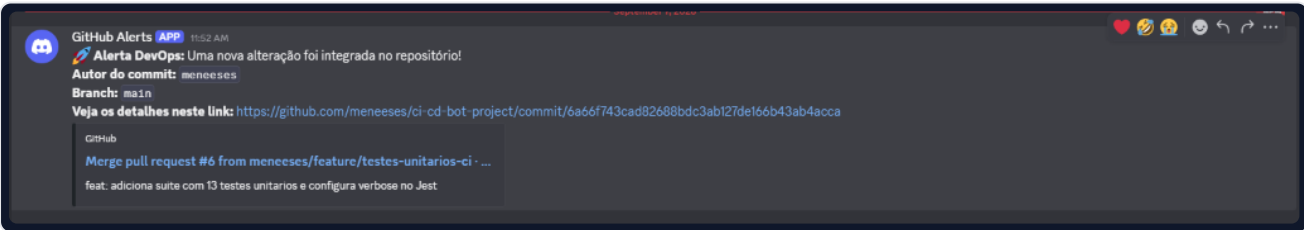


```
1 name: Alertas do Discord
2
3 on:
4   push:
5     branches:
6       - main # O workflow será acionado em cada push ou merge na branch main
7
8 jobs:
9   notify:
10    runs-on: ubuntu-latest
11    steps:
12      - name: Enviar notificação para o Discord
13        run: |
14          curl -X POST -H "Content-Type: application/json" \
15            -d '{
16              "content": "**Alerta DevOps:** Uma nova alteração foi integrada no repositório!\n**Autor do commit:** `${{ github.actor }}`\n**Branch:** ${{ github.ref }}"
17            }' \
18            ${ secrets.DISCORD_WEBHOOK_URL }
```

Descrição: Configuração do workflow acionado a cada evento de push/merge na branch main, disparando uma requisição POST com payload JSON para o Webhook do Discord via secrets.DISCORD_WEBHOOK_URL.

2. Atividade da Semana 6 - Notificação de Alerta Recebida no Discord

CANAL DO DISCORD COM NOTIFICAÇÃO AUTOMATIZADA



Descrição: Mensagem recebida no canal do Discord através da integração via Webhook após a realização do Merge da Pull Request #6 na branch main, informando o autor, branch e link direto para o commit.

3. Atividade Desta Semana - Código-Fonte dos Testes Unitários (Parte 1/3)

ARQUIVO: TESTS/BOT.TEST.JS (LINHAS 1 A 34)

```
botTest.js x
tests > botTest.js > describe('Suite de Testes Unitários do Bot CI/CD', () => { test('1. formatWelcomeMessage: deve formatar mensagem de boas-vindas com o nome do usuário') callback
1  const {
2    validateToken,
3    formatWelcomeMessage,
4    formatHelpMessage,
5    calculate,
6    formatUptime,
7    parseCommand,
8    processMessage
9  } = require('../src/bot');
10
11 describe('Suite de Testes Unitários do Bot CI/CD', () => {
12
13   // Teste 1: Validação de Boas-Vindas com nome personalizado
14   test('1. formatWelcomeMessage: deve formatar mensagem de boas-vindas com o nome do usuário', () => {
15     const message = formatWelcomeMessage('Carlos');
16     expect(message).toBe('Olá, Carlos! Bem-vindo ao bot de CI/CD da PUCPR.');
```

Descrição: Importação das funções modulares e definição dos Testes Unitários 1, 2 e 3 (validações de boas-vindas com e sem parâmetro, além de validação de rejeição de tokens inválidos).

4. Atividade Desta Semana - Código-Fonte dos Testes Unitários (Parte 2/3)

ARQUIVO: TESTS/BOT.TEST.JS (LINHAS 35 A 67)

```
botTest.js x
tests > botTest.js > describe('Suite de Testes Unitários do Bot CI/CD', () => { test('1. formatWelcomeMessage: deve formatar mensagem de boas-vindas com o nome do usuário') callback
11 describe('Suite de Testes Unitários do Bot CI/CD', () => {
35   // Teste 4: Validação de Token Telegram (formato válido)
36   test('4. validateToken: deve aceitar token no formato padrão do Telegram', () => {
37     // Padrão Telegram: 8 a 10 dígitos, seguido de ":" e 35 caracteres alfanuméricos/hífens
38     const validToken = '123456789:ABCDEFGHIJKLMNOpqrsTUVWxyz123456789';
39     expect(validateToken(validToken)).toBe(true);
40   });
41
42   // Teste 5: Parsing de comandos e argumentos
43   test('5. parseCommand: deve separar corretamente o comando e seus argumentos', () => {
44     const result = parseCommand('/calc 10 + 20');
45     expect(result.command).toBe('/calc');
46     expect(result.args).toEqual(['10', '+', '20']);
47   });
48
49   // Teste 6: Parsing de texto vazio ou inválido
50   test('6. parseCommand: deve lidar corretamente com entradas vazias ou nulas', () => {
51     expect(parseCommand('')).toEqual({ command: '', args: [] });
52     expect(parseCommand(null)).toEqual({ command: '', args: [] });
53   });
54
55   // Teste 7: Operações aritméticas básicas
56   test('7. calculate: deve realizar operações aritméticas básicas (+, -, *, /)', () => {
57     expect(calculate(10, '+', 5)).toBe(15);
58     expect(calculate(10, '-', 4)).toBe(6);
59     expect(calculate(6, '**', 7)).toBe(42);
60     expect(calculate(20, '/', 4)).toBe(5);
61   });
62
63   // Teste 8: Tratamento de divisão por zero
64   test('8. calculate: deve retornar mensagem de erro amigável ao tentar dividir por zero', () => {
65     const result = calculate(10, '/', 0);
66     expect(result).toBe('Erro: Divisão por zero não é permitida');
```

Descrição: Definição dos Testes Unitários 4 a 8 (validação de formato de token do Telegram, parsing de comandos com argumentos, operações aritméticas e tratamento de divisão por zero).

5. Atividade Desta Semana - Código-Fonte dos Testes Unitários (Parte 3/3)

ARQUIVO: TESTS/BOT.TEST.JS (LINHAS 68 A 101)

```
bottest.js x
tests > bottest.js > describe('Suite de Testes Unitários do Bot CI/CD', () => { test('1. formatWelcomeMessage: deve formatar mensagem de boas-vindas com o nome do usuário') callback
11 describe('Suite de Testes Unitários do Bot CI/CD', () => {
69 // Teste 9: Tratamento de operador inválido e entradas não numéricas
70 test('9. calculate: deve retornar erro para operador inválido ou entradas que não são números', () => {
71   expect(calculate(10, '^', 2)).toBe('Erro: Operador '^' não suportado');
72   expect(calculate('abc', '+', 5)).toBe('Erro: Números inválidos');
73 });
74
75 // Teste 10: Formatação de tempo de execução (uptime)
76 test('10. formatUptime: deve formatar segundos em representação legível (horas, minutos e segundos)', () => {
77   expect(formatUptime(45)).toBe('45s');
78   expect(formatUptime(125)).toBe('2m 5s');
79   expect(formatUptime(3665)).toBe('1h 1m 5s');
80   expect(formatUptime(-10)).toBe('0s');
81 });
82
83 // Teste 11: Processamento de mensagem - Comando /help
84 test('11. processMessage: deve retornar texto de ajuda para o comando /help', () => {
85   const reply = processMessage('/help');
86   expect(reply).toContain('🌟 Comandos disponíveis:');
87   expect(reply).toContain('/start');
88   expect(reply).toContain('/calc');
89 });
90
91 // Teste 12: Processamento de mensagem - Comando /echo
92 test('12. processMessage: deve responder com o texto replicado no comando /echo', () => {
93   const reply = processMessage('/echo Teste de Integração Contínua');
94   expect(reply).toBe('Teste de Integração Contínua');
95 });
96
97 // Teste 13: Processamento de mensagem - Comando desconhecido
98 test('13. processMessage: deve responder com mensagem de comando não reconhecido para comandos inválidos', () => {
99   const reply = processMessage('/comando_inexistente');
100   expect(reply).toBe('Comando não reconhecido. Digite /help para ver os comandos disponíveis.');
```

Descrição: Definição dos Testes Unitários 9 a 13 (tratamento de operadores inválidos, cálculo de uptime, processamento dos comandos /help e /echo, e tratamento de comandos não reconhecidos). Total de 13 testes unitários.

6. Atividade Desta Semana - Execução dos Testes na Pull Request (GitHub)

PULL REQUEST #6: FEATURE/TESTES-UNITARIOS-CI -> MAIN

feat: adiciona suite com 13 testes unitarios e configura verbose no Jest #6

Merged meneeses merged 1 commit into main from feature/testes-unitarios-ci

Conversation 0 Commits 1 Checks 0 Files changed 4 +273 -14

meneeses commented 1 minute ago

No description provided.

Mention @copilot in a comment to make changes to this pull request.

feat: adiciona suite com 13 testes unitarios e configura verbose no Jest

Copilot AI lite review requested due to automatic review settings

Copilot started reviewing on behalf of meneeses

meneeses merged commit 6a66f74 into main

1 of 2 checks passed

Pull request successfully merged and closed

You're all set — the feature/testes-unitarios-ci branch can be safely deleted.

Reviewers: Copilot Lite

Still in progress? Convert to draft

Assignees: No one—assign yourself

Labels: None yet

Projects: None yet

Milestone: No milestone

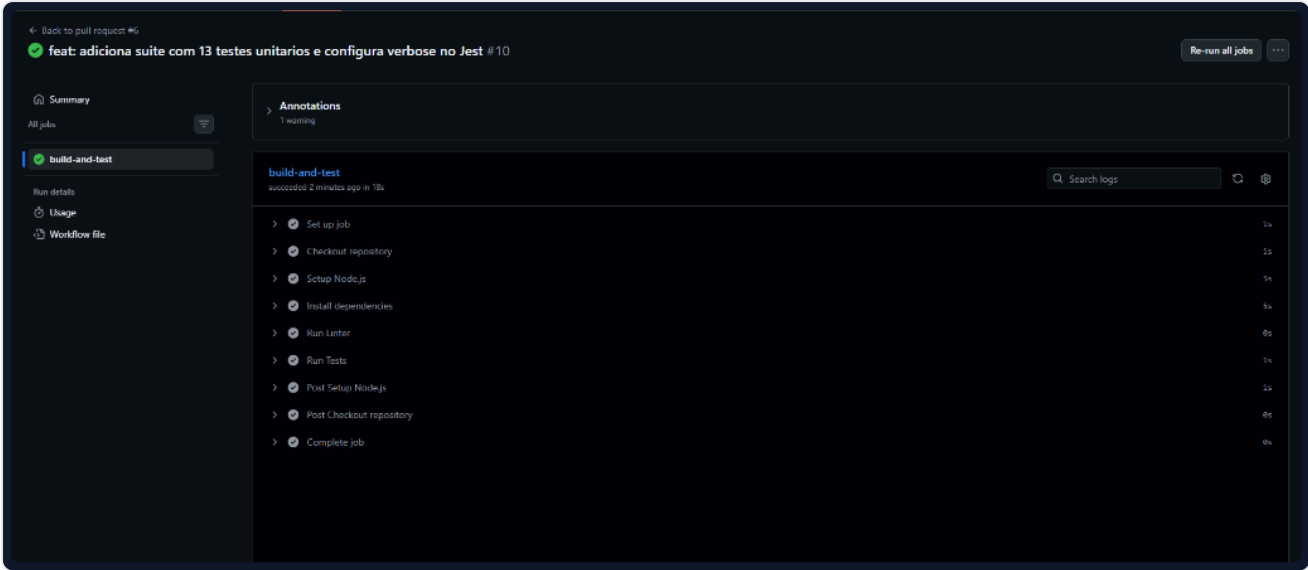
Development: Successfully merging this pull request may close these issues.

Notifications Customize

Descrição: Visão geral da Pull Request no GitHub demonstrando o status de aprovação com check verde no commit e a realização do merge para a branch principal.

7. Atividade Desta Semana - Detalhes do Workflow de CI no GitHub Actions

EXECUÇÃO BEM-SUCEDIDA DO JOB 'BUILD-AND-TEST'



Descrição: Comprovação de execução de todas as etapas do workflow de Integração Contínua disparado na PR: Setup Node.js, Install dependencies, Run Linter e Run Tests (todos concluídos com sucesso).